

GPUs in HPC systems Architecture and Programming model

Ernesto Sanchez

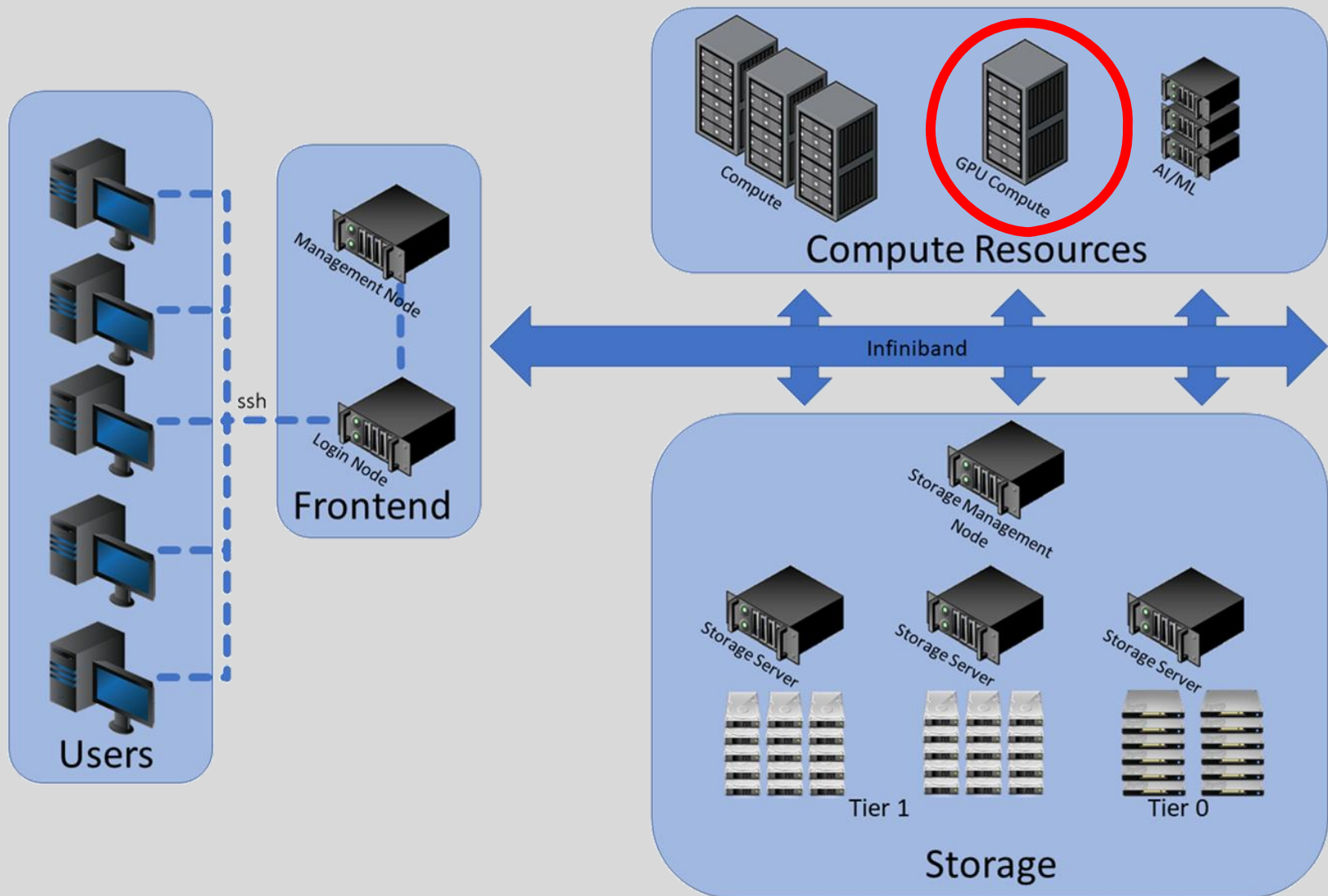
GPUs in HPC systems

- Hardware accelerators in HPC machines
- GPUs in HPC systems
- Overview of the GPU architecture
- Programming model of GPUs
- Basic CUDA programming model.

Acknowledgement

The following slides were prepared by prof. Esteban Rodriguez.

A general overview of an HPC system



Hardware Accelerators

Special purpose designs (chips) to support the processor for accelerating a specific function or workload (“co-processors”).

- **Concurrency** (Flynn’s taxonomy)
 - Sequences (or streams/threads)
 - Data
 - Instructions (commands, operations)
- **Single-Instruction Multiple-Data** (SIMD)
- **Multiple-Instruction Multiple-Data** (MIMD)

Very relevant components in HPC machines since 90’s.

Hardware Accelerators

Types:

- APU: Accelerated Processing Unit
- DPU: Deep Learning Processor Unit
- FPGA: Field-Programmable Gate-Array
- GPU: Graphics Processing Unit
- GPGPU: General-Purpose Graphics Processing Unit
- IPU: Intelligence Processing Unit
- LPU: Language Processing Unit
- VPU: Vector Processing Unit
- TPU: Tensor Processing Unit

GPUs in HPC systems

CPU: Use for primary calculations

- General-purpose capabilities (sequential operations)
- Established technology
- Equipped with 8 or fewer, powerful cores
- Optimal for some types of concurrent processes
- Not able to support large-scale parallel computations

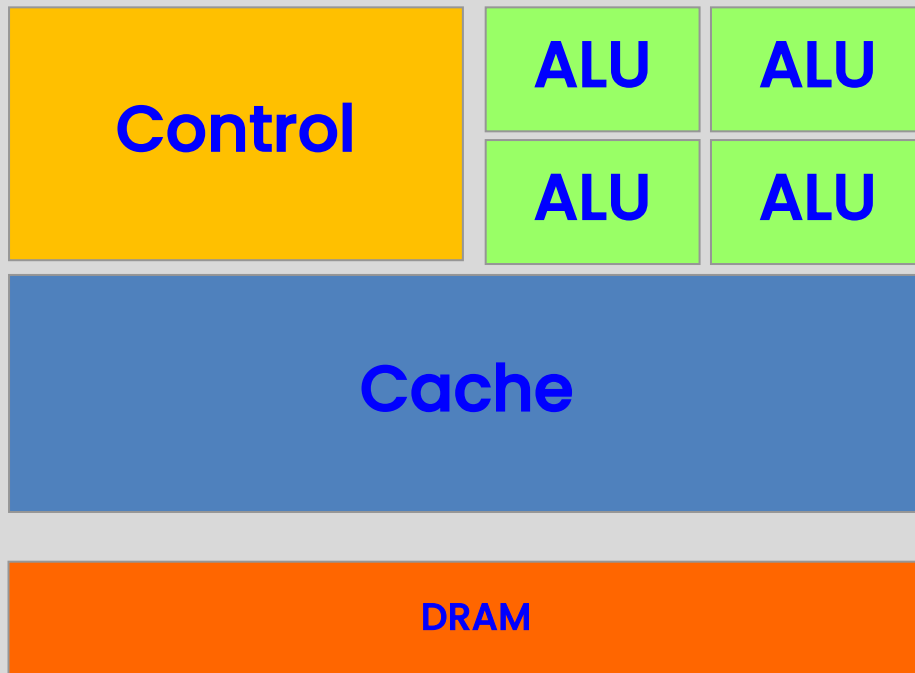
GPU/GPGPU: Use in special operations (accelerate performance)

- Designed for parallelizable problems (Graphics)
- Became capable of general computations (HPC)
- Equipped with hundreds/thousands of simple cores
- Needs a host/master to control its execution.

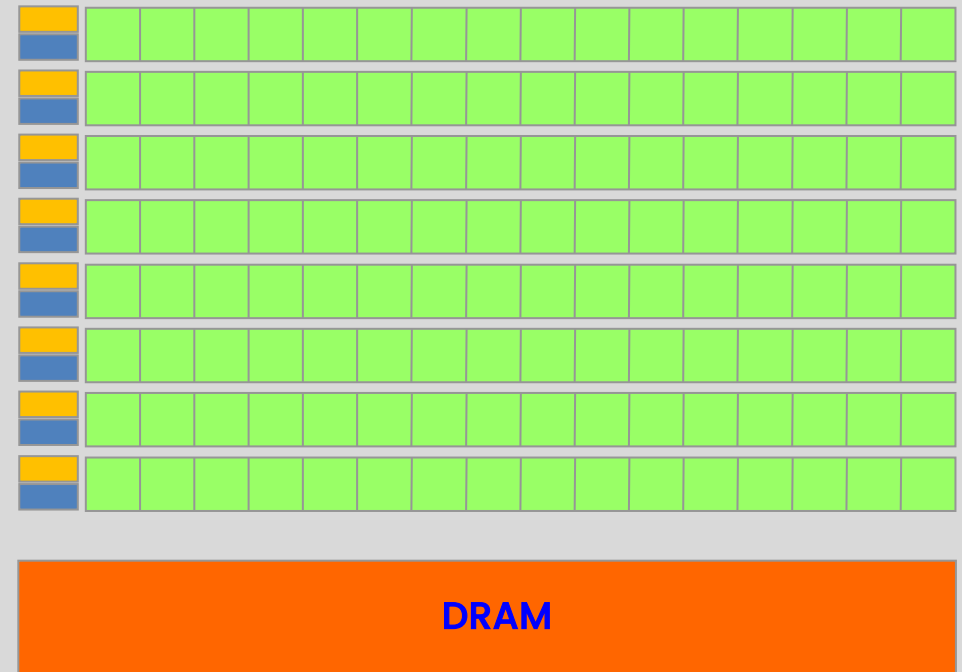
GPU definition

A GPU is a device equipped with several highly parallel microprocessors (thousands of cores) and a private memory with very high bandwidth (about 1000GB/s).

CPU



GPU

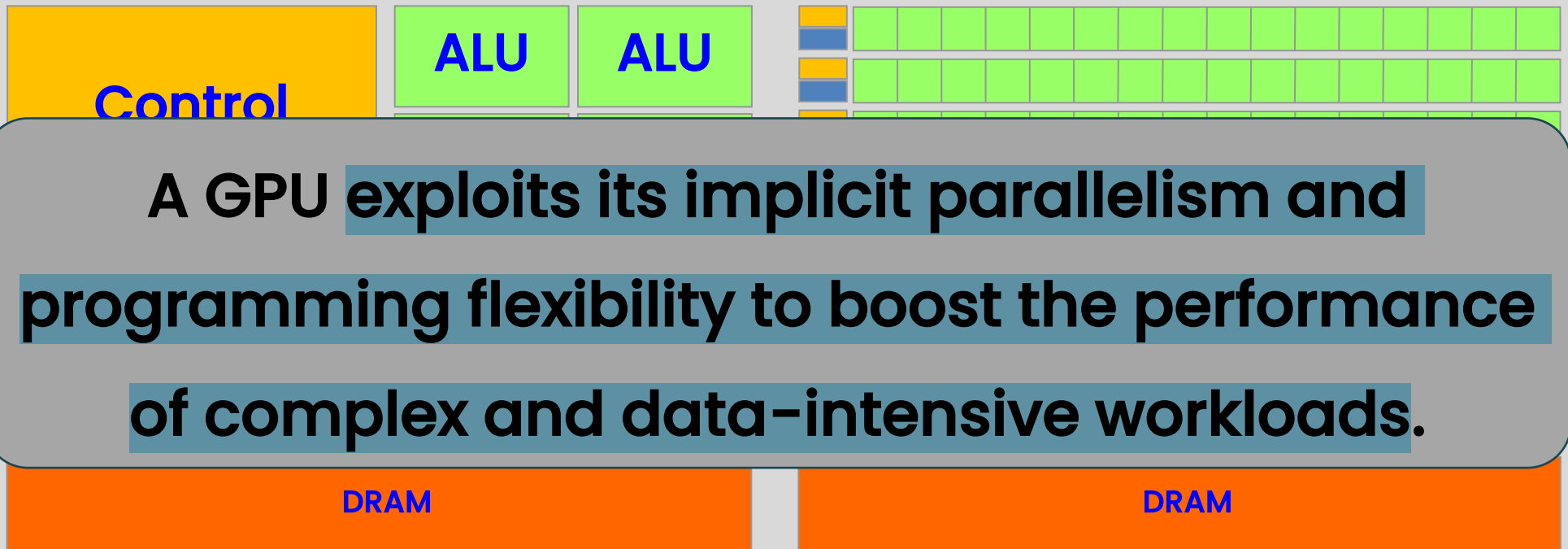


GPU definition

A GPU is a device equipped with several highly parallel microprocessors (thousands of cores) and a private memory with very high bandwidth (about 1000GB/s).

CPU

GPU



Rendering complex scenes

GPUs were originally designed to render complex 3D scenes composed of millions of data points/vertex at high frame rates (60–120 FPS).

The rendering process: transformations based on linear algebra operations and (mostly local) filters.

- The same set of operations are applied on each data point of a scene
- Each operation is independent with respect to data
- All operations are performed in **parallel** using a huge number of **threads** which process all data independently.

Thread definition

A Thread is defined as an independent execution flow of a program (*Kernel*), which share the same memory resources of the main process (access to main data).

A process can spawn multiple threads of execution, each can follow an independent flow.

Threads inside a process can:

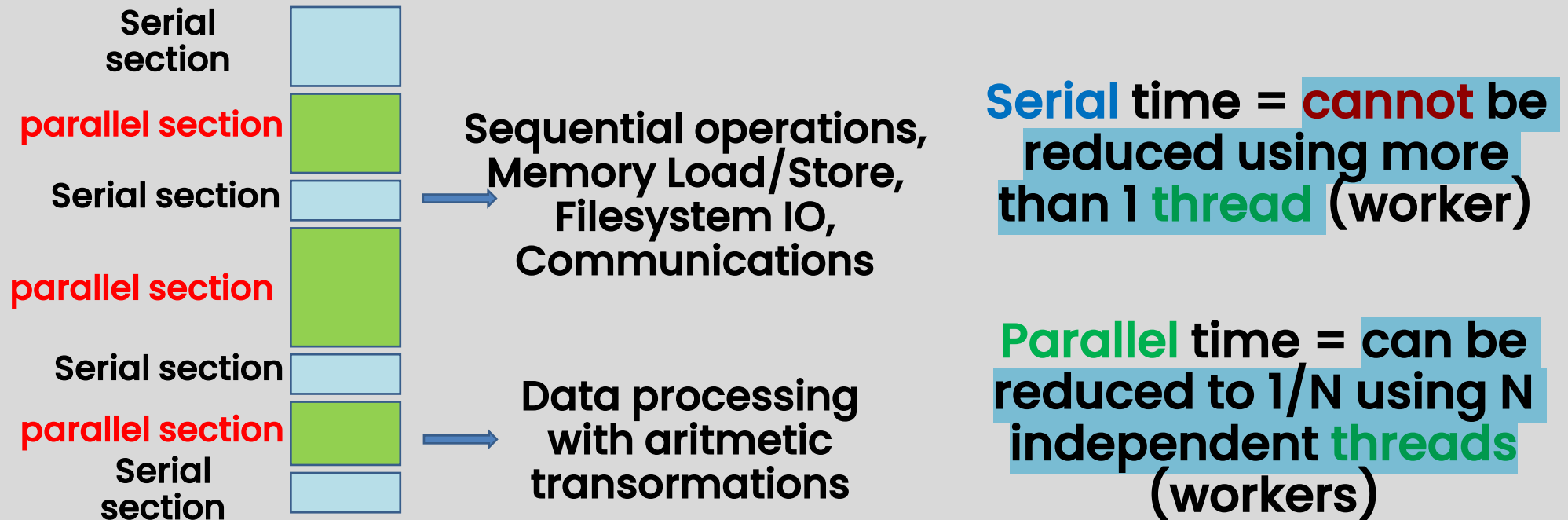
- Exchange data since they share the same program's memory
- Use local memory not shared among threads
- Synchronize with other threads (waiting dependences).

GPUs in HPCs

Amdahl's Law

Program Execution Time

$$T_w = \text{sum}(\text{serial (sequential) time} + \text{parallel time})$$

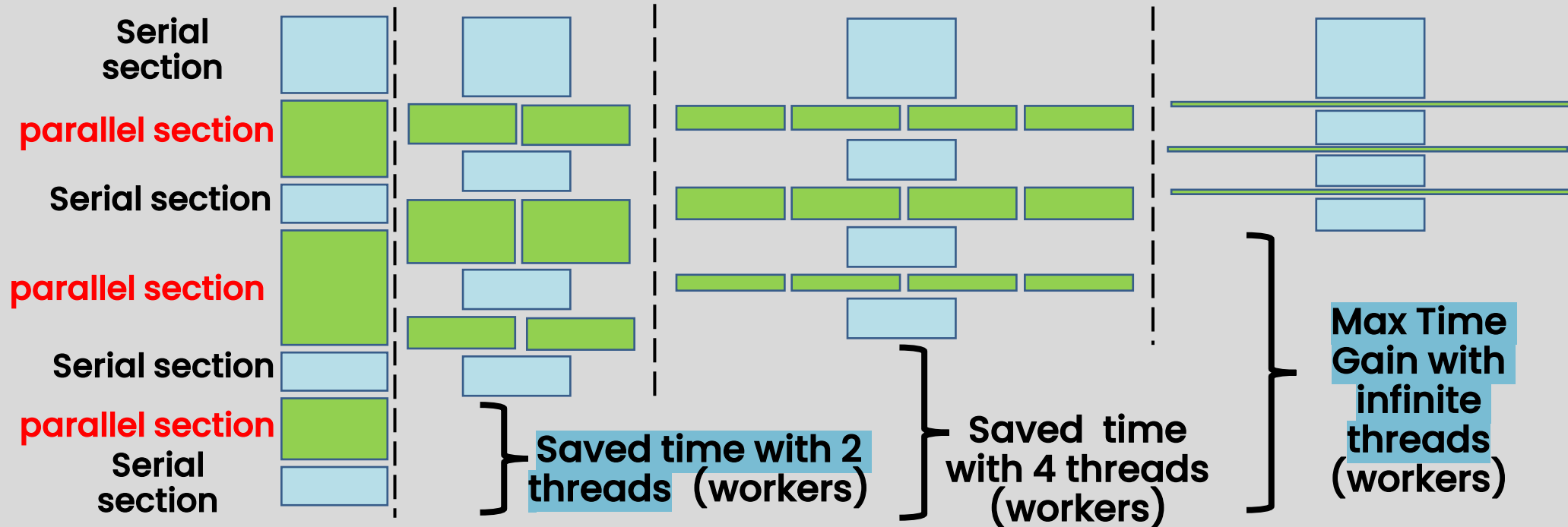


GPUs in HPCs

Amdahl's Law

Program Execution Time

$$T_w = \text{sum}(\text{serial (sequential) time} + \text{parallel time})$$



GPUs in HPCs

Parallelism of a Single-Program Multiple-Data

```
...  
// typical loop over each point with the same set of operations  
For_each point in collection_of_points do:  
  output_data = transformations_on_point(point, input_data)  
...
```

- If the **transformations** can be applied independently on each point, the output result is independent on the order of point computation
- If **transformations** are independent, we can speed up the elaboration using parallel work:
 - Apply the same transformation (Single Program) ...
 - ... to each point (Multiple Data)
 - ... using **multiple threads** concurrently

GPUs in HPCs

Parallelism of a Single-Program Multiple-Data

Example:

- Addition of two vectors A and B

A

a_0	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9
-------	-------	-------	-------	-------	-------	-------	-------	-------	-------

+

B

b_0	b_1	b_2	b_3	b_4	b_5	b_6	b_7	b_8	b_9
-------	-------	-------	-------	-------	-------	-------	-------	-------	-------

=

results

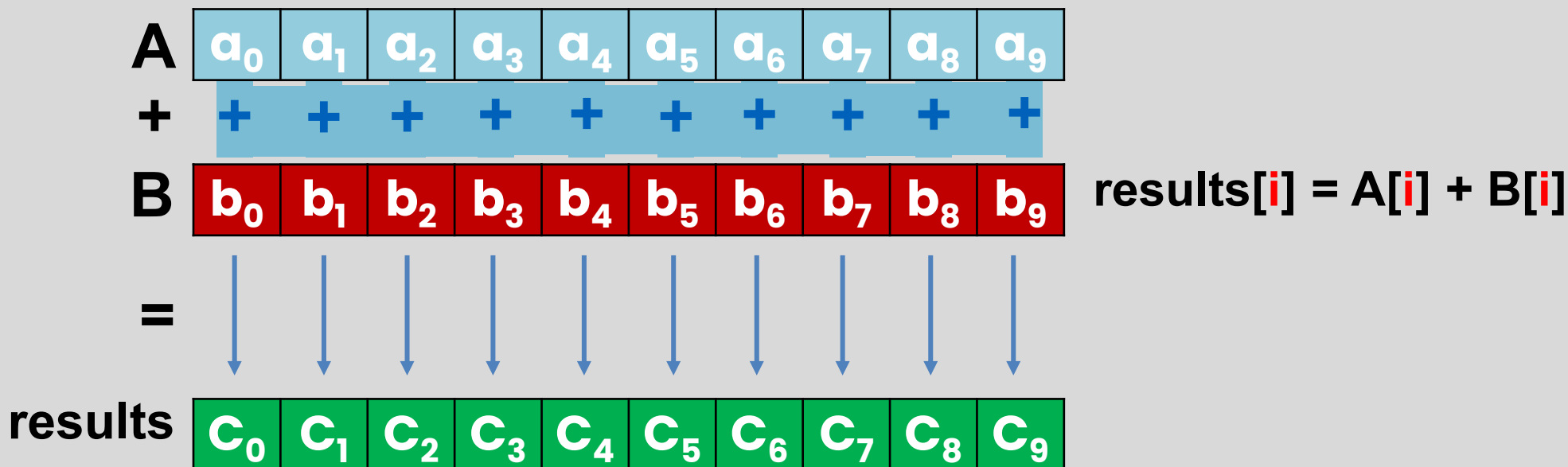
c_0	c_1	c_2	c_3	c_4	c_5	c_6	c_7	c_8	c_9
-------	-------	-------	-------	-------	-------	-------	-------	-------	-------

GPUs in HPCs

Parallelism of a Single-Program Multiple-Data

Example:

- Addition of two vectors A and B



GPUs in HPCs

Parallelism of a Single-Program Multiple-Data Example:

- Addition of two vectors A and B

C (sequential) code

```
...  
for (int i = 0; i < n; i++)  
    result[i] = A[i] + B[i];  
...
```



i = 0
i = 1
i = 2
...

GPUs in HPCs

Parallelism of a Single-Program Multiple-Data

Example:

- Addition of two vectors A and B

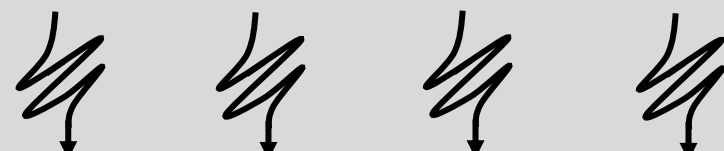
C (sequential) code

```
...  
for (int i = 0; i < n; i++)  
    result[i] = A[i] + B[i];  
...
```

Parallel code

```
...  
int i[];  
index_init(i[]);  
// i(0) = 0, i(1) = 4, i(2) = 8...  
result[i] = A[i] + B[i];  
...
```

 $i = 0$
 $i = 1$
 $i = 2$
...

 ...
 $i(0) = 0$ $i(1) = 4$ $i(2) = 8$ $i(3) = 12$

GPUs in HPCs

Parallelism of a Single-Program Multiple-Data

Example:

- Addition of two vectors A and B

C (sequential) code

```
...  
for (int i = 0; i < n; i++)  
    result[i] = A[i] + B[i];  
...
```

Parallel code

```
...  
int i[]; HARDWARE OR SOFTWARE?  
index_init(i[]);  
// i(0) = 0, i(1) = 4, i(2) = 8...  
result[i] = A[i] + B[i];  
...
```

 $i = 0$
 $i = 1$
 $i = 2$
...

 $i(0) = 0$  $i(1) = 4$  $i(2) = 8$  $i(3) = 12$...

GPUs in HPCs

GPU threads

GPU can manage thousands of threads efficiently

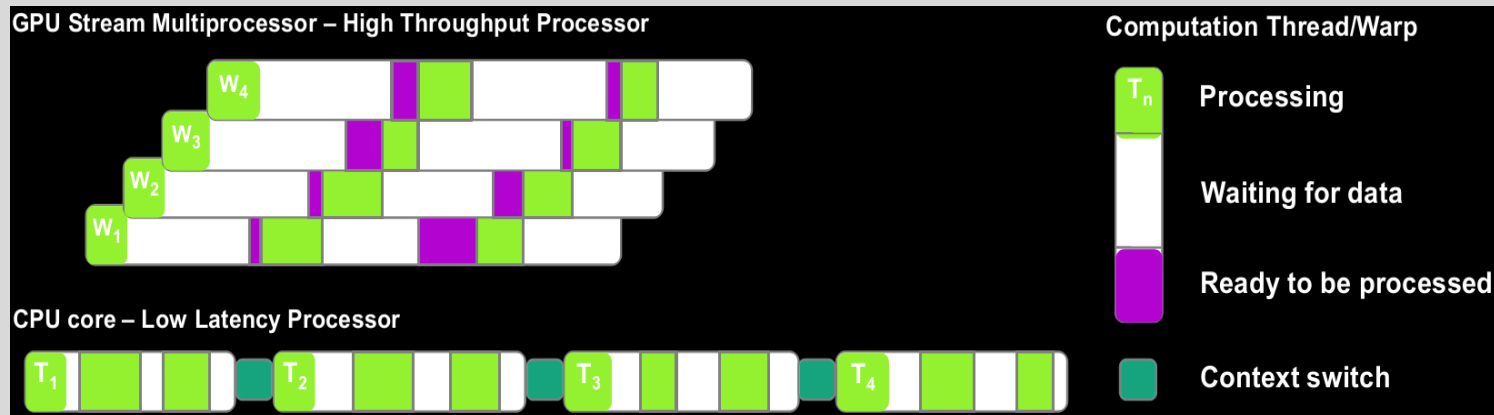
- GPU threads are **extremely light weighted**
 - GPU have thousands of available registers
 - Each thread running on a GPU maintains its status in its own registers
 - No penalty in case of a context-switch

GPUs in HPCs

GPU threads

GPU can manage thousands of threads efficiently

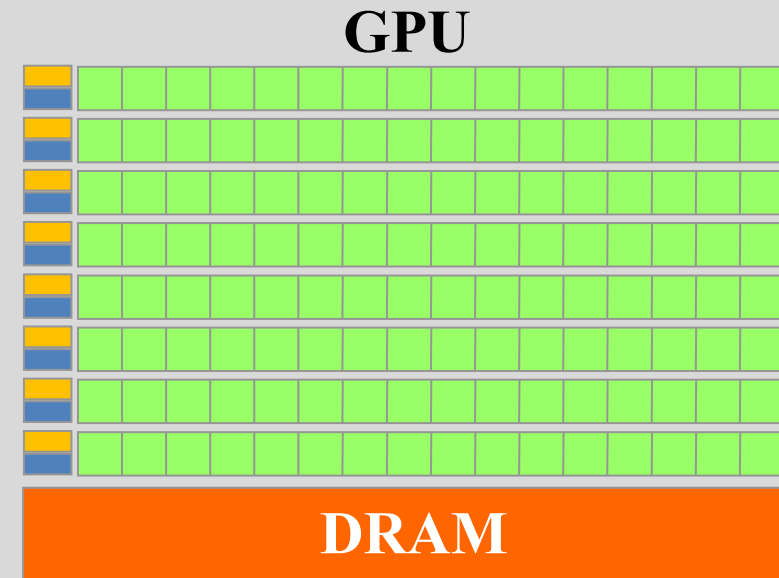
- GPU threads are **extremely light weighted**
 - GPU have thousands of available registers
 - Each thread running on a GPU maintains its status in its own registers
 - No penalty in case of a context-switch
- The more threads are active, the more the GPU hardware can hide memory or computational latencies



Overview of the GPU ARCHITECTURE

GPU organization

GPU hardware dedicate more silicon for ALU and registers, hiding memory and instruction latencies with computing operations



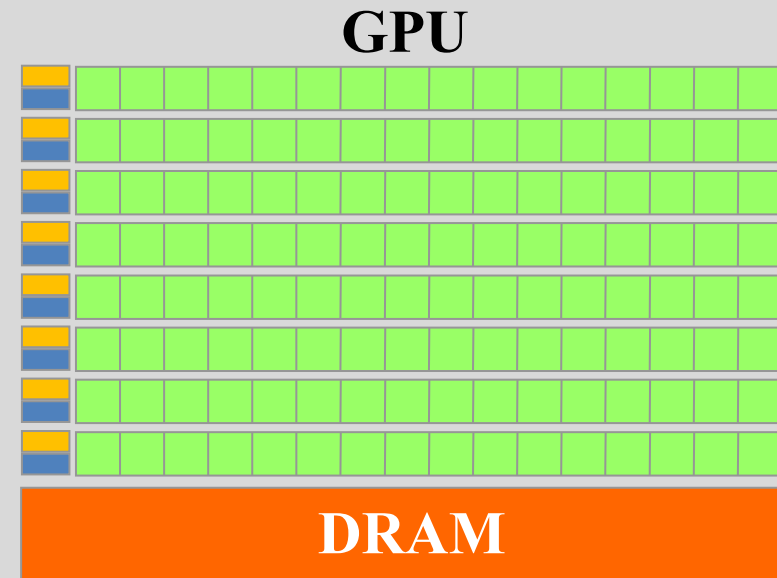
Overview of the GPU ARCHITECTURE

GPU organization

GPU hardware dedicate more silicon for ALU and registers, hiding memory and instruction latencies with computing operations

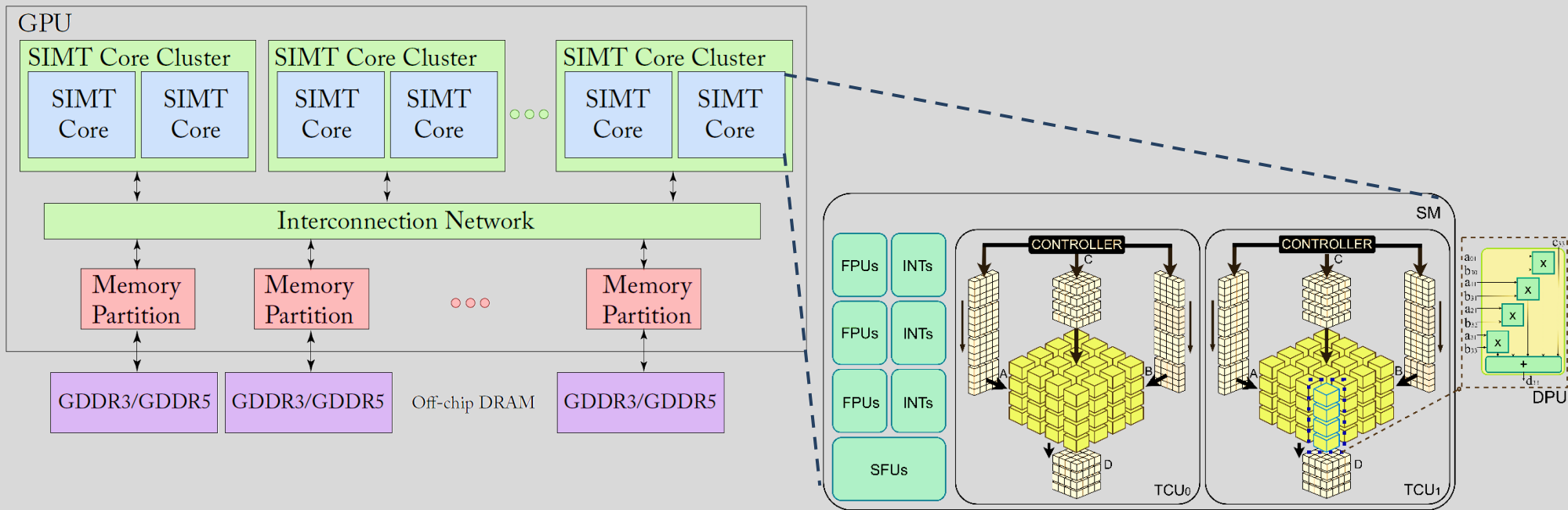
Main Global Memory

- medium size (16-40 GB)
- very high bandwidth (800-1200 GB/s)
- **Streaming Multiprocessors (SM / SIMT)**
 - grouping independent cores and control units
- Each SM unit has
 - many stream cores (> 100 cores)
 - lots of registers (32K-64K)
 - instruction scheduler dispatchers
 - a shared memory with very fast access to data



Overview of the GPU ARCHITECTURE

GPU organization



GPU ARCHITECTURE

GPU organization

INT32: performs 32-bit add, multiply, multiply-add, and maybe some logical operations.

FP32: performs 32-bit floating-point add, multiply, multiply-add, and similar operations.

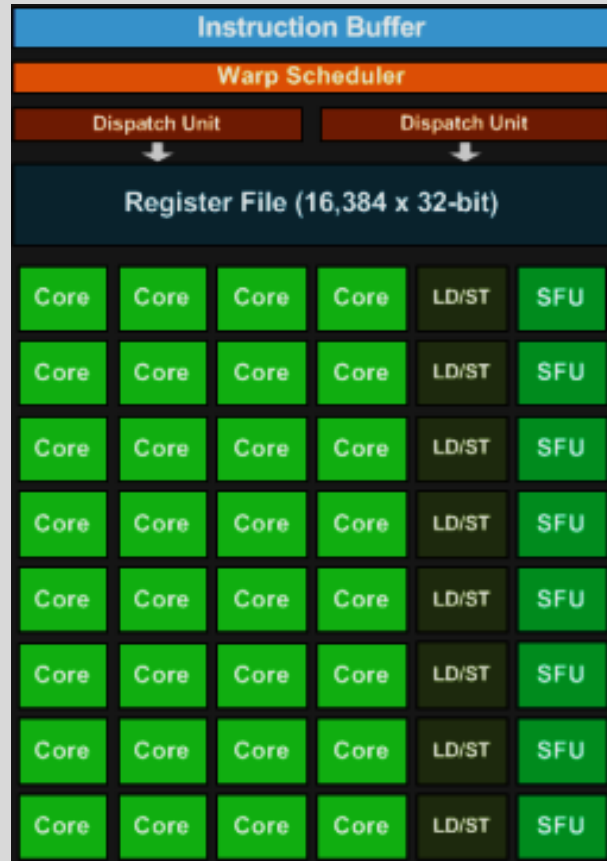
FP64: executes 64-bit FP operations.

Special Functional Unit (SFU): performs reciprocal ($1/x$) and transcendental instructions, such as: sine, cosine, and reciprocal square root.

Load/Store (LS): performs loads and stores from shared, constant, local, and global memory address spaces.

Tensor Core: specialized units to compute $A*B+C$ matrix product.

GPU ARCHITECTURE



**Fermi SM
(2011).**



GPU ARCHITECTURE

Example of GPU architecture

- SM composed of two independent blocks
- Each block comprises:
 - 1 warps x 2 dispatchers
 - 32 ALU SIMD units
 - **16 FP64 units**
 - 8 Load/Store units
 - 8 SFU units
 - 32,768 32bits registers
- each block accesses:
 - 64KB shared memory
 - L1 64KB cache
 - 4 texture units



Streaming Multiprocessor of NVIDIA Pascal
(2016)

GPU ARCHITECTURE

Example of GPU architecture

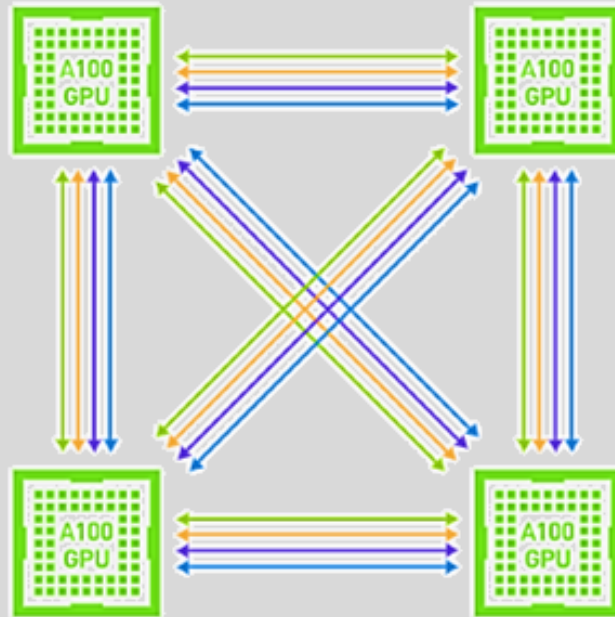
- SM with 4 independent blocks
- each block comprises:
 - 1 warps x 2 dispatchers
 - 16 FP32 + 16 INT32 (**simultaneous**)
 - 8 FP64 units
 - 2 Tensor Core units
 - 8 Load/Store units
 - 4 SFU units
 - 32,768 32bits registers
- each block accesses:
 - 192KB for L1/shared memory
 - 4 texture units
- **NVLink technology**
 - 300GB/s bandwidth (12X PCIe Gen3)



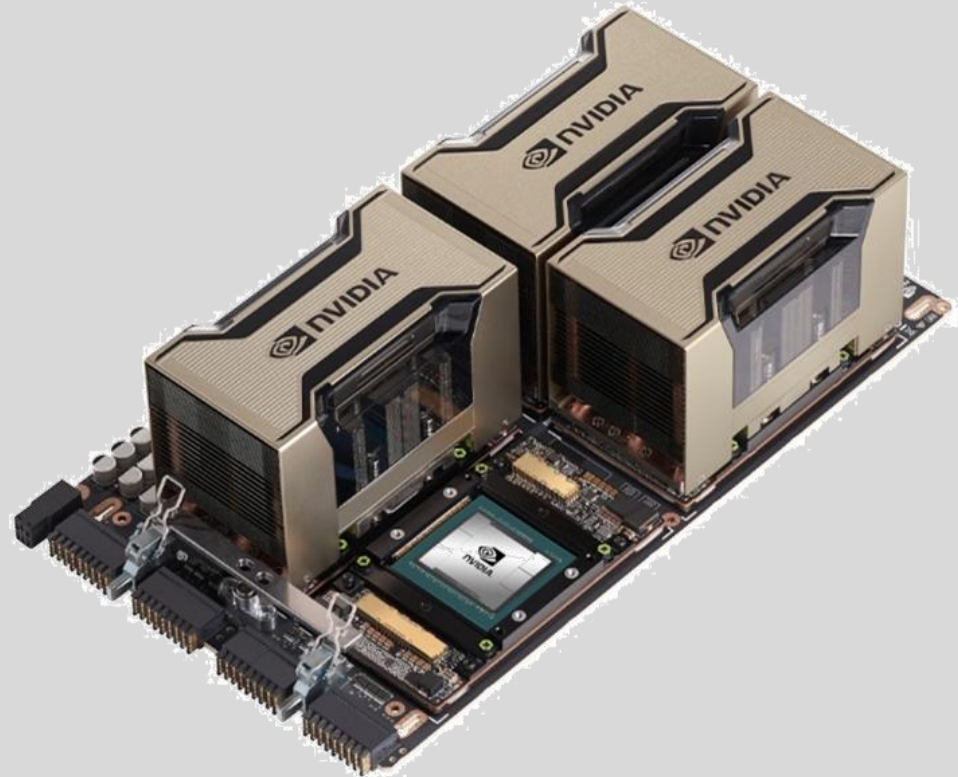
HPC class SM of NVIDIA Ampere (2020)

GPU ARCHITECTURES

HPC class GPU architecture

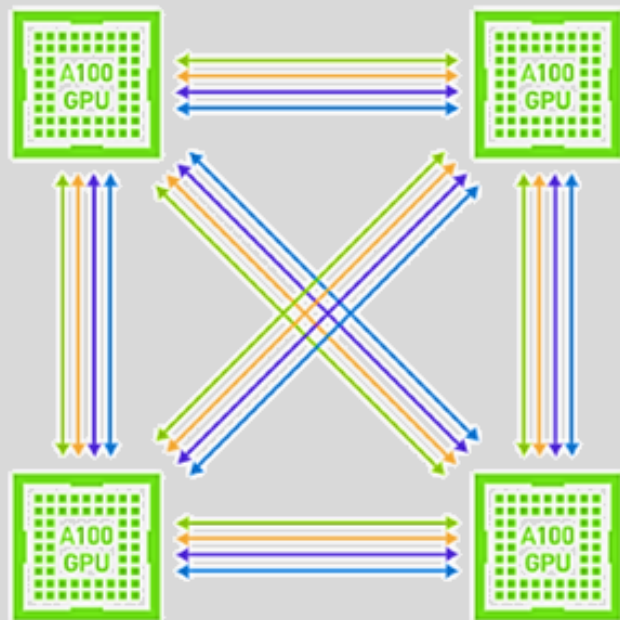


NVLink technology
Nvidia

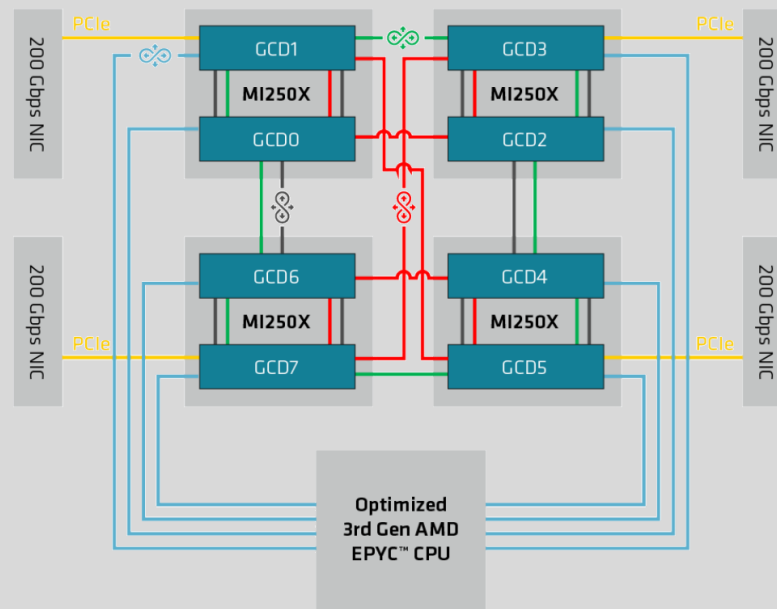


GPU ARCHITECTURES

HPC class GPU architecture



NVLink technology
Nvidia



Infinity technology
AMD

GPU ARCHITECTURES

HPC class GPU architecture

Nvidia

Model	FP32 [TFlops]	Cores	RAM [GB]	Bandwidth [GB/s]	Link
Kepler K40	4.3	2,280	12 GDDR5	240	PCIe 3.0 (15.8 GB/s)
Pascal P100	10.6	3,584	16 HBM2	720	PCIe 3.0 (15.8 GB/s)
Volta V100	15.7	5,120	16/32 HBM2	900	PCIe 3.0 (15.8 GB/s)
Ampere A100	19.5	8,192	40 HBM2	1,500	PCIe 4.0 (31.6 GB/s)
Blackwell B200	180	20,480	192 HBM3e	1,600	PCIe 6.0 (64 GB/s)

GPU ARCHITECTURES

HPC class GPU architecture

AMD

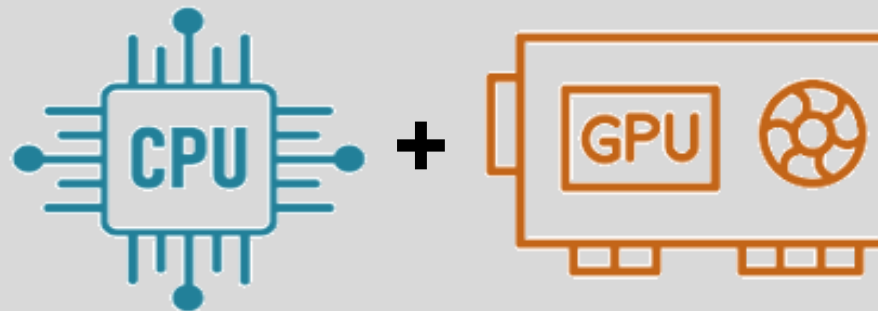
Model	FP32 [TFlops]	Cores	RAM [GB]	Bandwidth [GB/s]	Link
Radeon MI8	8.2	4,096	4 HBM	512	PCIe 3.0 (15.8 GB/s)
Radeon MI25	12.3	4,096	16 HBM2	484	PCIe 3.0 (15.8 GB/s)
Radeon MI50	13.4	3,840	16 HBM2	1,024	PCIe 3.0 (15.8 GB/s)
Radeon MI100	32.1	7,680	32 HBM2	1,200	PCIe 4.0 (31.6 GB/s)
Instinct MI300X	653.7	19,456	192 HBM3	5,300	PCIe 5.0 x16 (128 GB/s)

GPU Programming model

GPU Programming model

GPGPU Programming model

- General Purpose GPU Programming focuses on the use of GPU computational power to solve problems other than graphics
- **CPU** and **GPU** are separate devices with separate memory space addresses
- GPU is seen as an auxiliary coprocessor
- They should work together for best benefit and performances



GPU Programming model

GPGPU Programming model

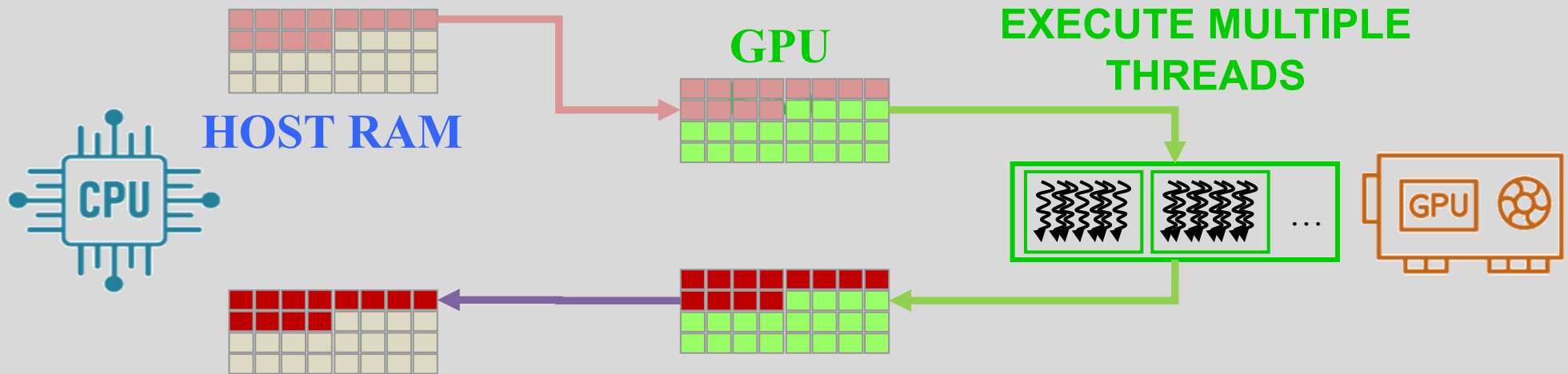
- Serial parts of a program, or those with low level of parallelism, keep running on the **CPU** (HOST)
- Computational-intensive data-parallel regions are executed on the **GPU** (device)
- Required data is moved on GPU memory and back to HOST memory



GPU Programming model

Data movement

- Data must be moved from HOST to DEVICE memory to be processed on the **GPU**
- When data is processed, and no more needed on the GPU, it is transferred back to HOST



GPU Programming model

Data movement

- Sometimes data transfer can take more than the GPU computation if the problem is too "easy"
- GPU are best suited for computational intensive problems, not just "parallel"
- for example:
 - Vector add is not suited for GPU
(order N)
 - Matrix-matrix multiplication is a good candidate for GPU
(order N^3)

GPU Programming model

Data movement

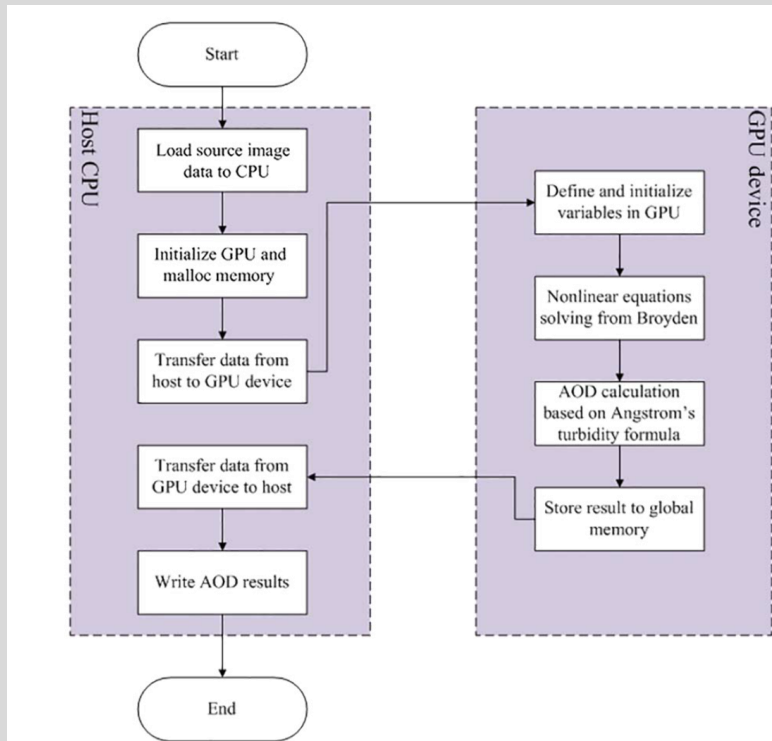


Fig. 5. Flowchart of the CUDA implementation.

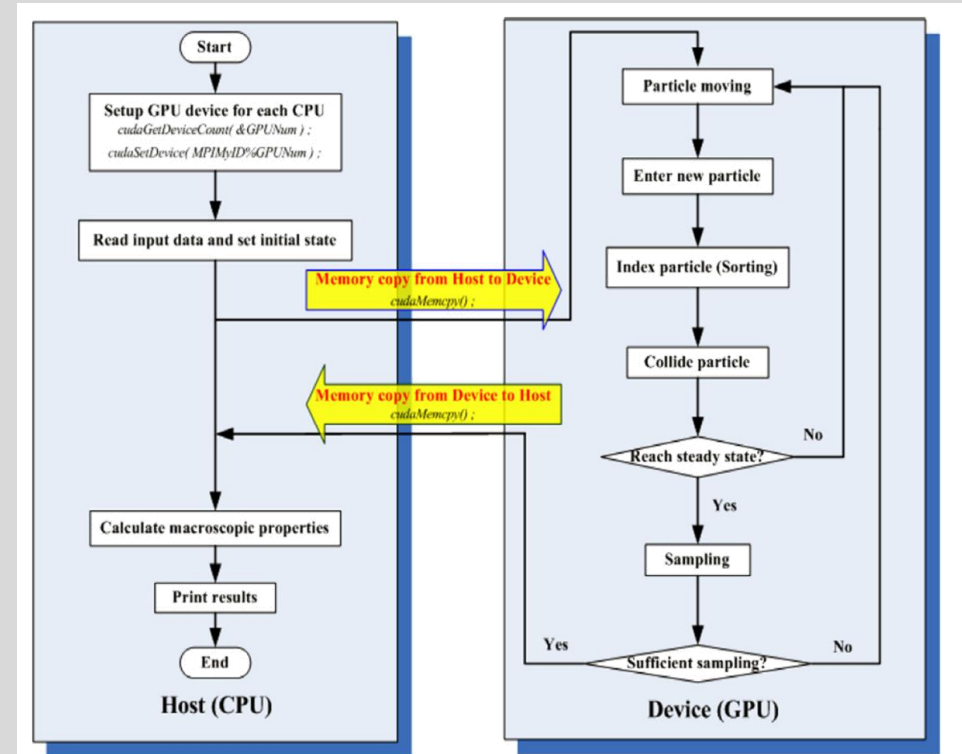


Fig. 2. Flowchart describing the application of DSMC to GPU-accelerated computation.

GPU Programming model

GPU Execution

- A function which runs on a GPU is called a “kernel”
- when a kernel is launched on a GPU, thousands of threads will execute its code
- programmer chooses the number of threads to launch
- each thread acts on a different data element independently.

GPU Programming model

GPU Execution

- Example: vector add

GPU Programming model

GPU Execution

- Example: vector add

```
void vecAddCPU (int N,  
               const float *A,  
               const float *B,  
               float *C) {  
    for(int i = 0; i < N; i++)  
        c[i] = a[i] + b[i];  
}  
...  
//call vecAddCPU on N elements  
vecAddCPU ( N, a, b, c );
```

GPU Programming model

GPU Execution

- Example: vector add

```
void vecAddCPU (int N,  
               const float *A,  
               const float *B,  
               float *C) {  
    for(int i = 0; i < N; i++)  
        c[i] = a[i] + b[i];  
}  
...  
//call vecAddCPU on N elements  
vecAddCPU ( N, a, b, c );
```

```
void vecAddGPU (int N,  
               const float *A,  
               const float *B,  
               float *C) {  
    int i = ... //use unique thread  
               index  
    if(i < N) c[i] = a[i] + b[i];  
}  
...  
//copy data from host to device  
//launch kernel with 1 block of  
N threads  
vecAddGPU<<<1,N>>>(N,a,b,c);  
// copy data from device to host
```

GPU Programming model

GPU Execution

- Example: vector add

One thread

```
void vecAddCPU (int N,  
               const float *A,  
               const float *B,  
               float *C) {  
    for(int i = 0; i < N; i++)  
        c[i] = a[i] + b[i];  
}  
...  
//call vecAddCPU on N elements  
vecAddCPU ( N, a, b, c );
```

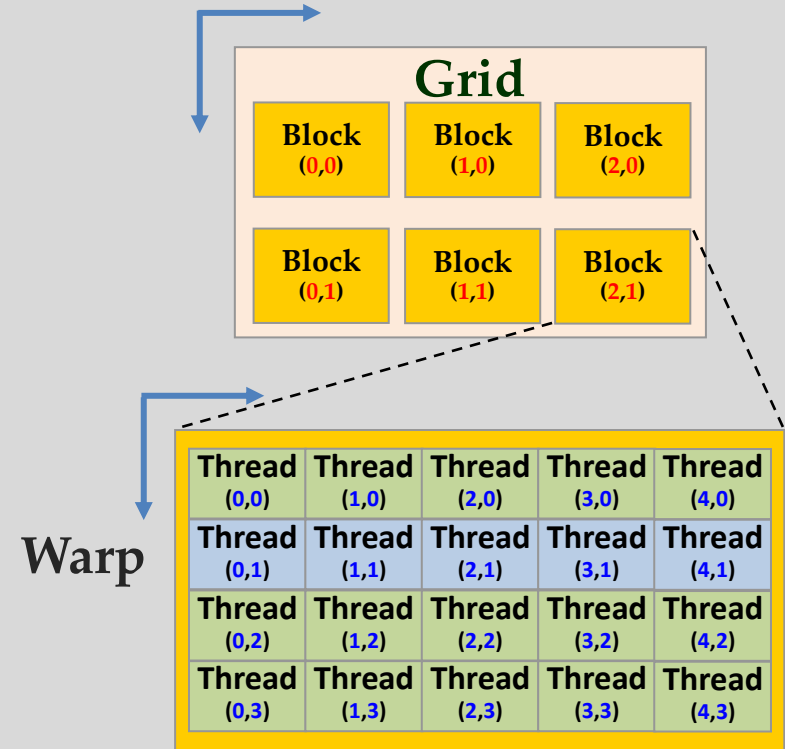
N threads

```
void vecAddGPU (int N,  
               const float *A,  
               const float *B,  
               float *C) {  
    int i = ... //use unique thread  
               index  
    if(i < N) c[i] = a[i] + b[i];  
}  
...  
//copy data from host to device  
//launch kernel with 1 block of  
N threads  
vecAddGPU<<<1,N>>>>(N,a,b,c);  
// copy data from device to host
```

GPU Programming model

GPU Thread hierarchy

- To compute N elements (one per thread!) in parallel on a GPU, at least N concurrent threads must be created on the device
- GPU threads are grouped together in teams or “thread blocks”
- Threads belonging to the same block or team can cooperate together exchanging data through a shared memory area.

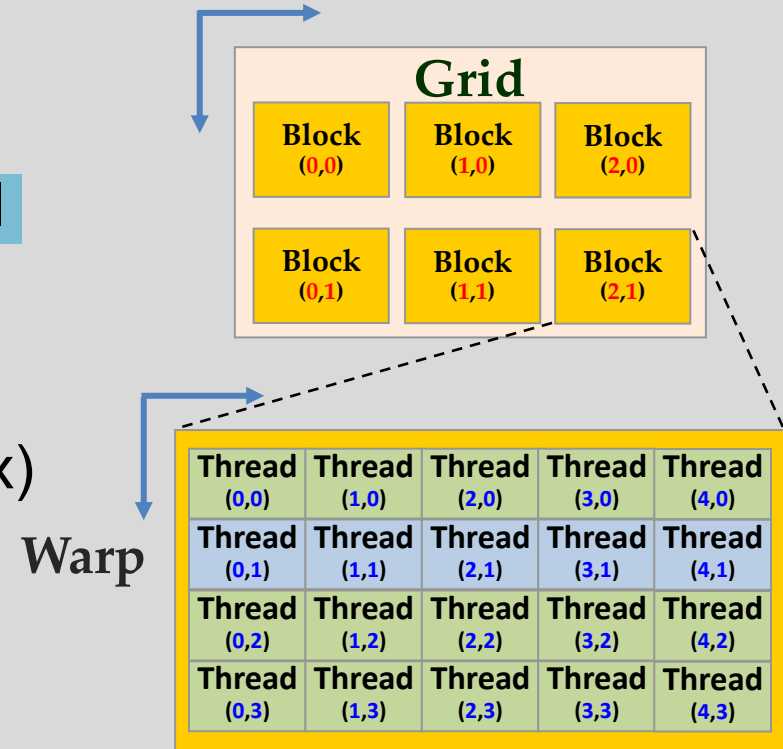


GPU Programming model

GPU Thread hierarchy

Thread

- The smallest execution unit.
- Executes a single instance of a kernel (function)
- Each thread has:
 - Its own registers
 - Its own thread ID (e.g., threadIdx.x)
- Threads typically process one data element (e.g., one pixel, one array entry).



GPU Programming model

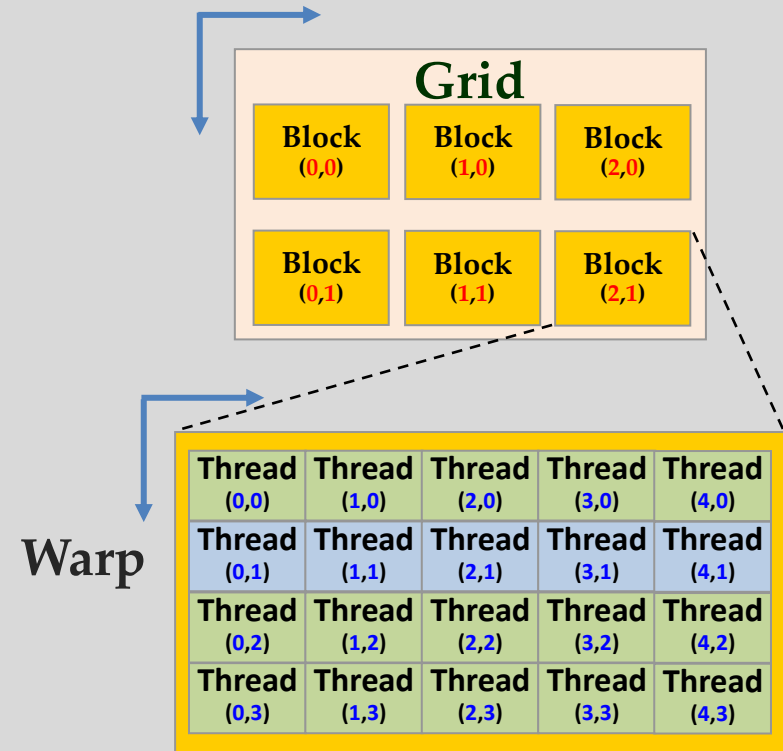
GPU Thread hierarchy

Warp

- A hardware-level concept (not visible explicitly in standard programming).
- A warp is a group of threads executed simultaneously in SIMT fashion.
- Typical size (NVIDIA GPUs): 32 threads per warp

All threads in a warp:

- Execute the same instruction at the same time
- Divergence (if-else) causes serialization.

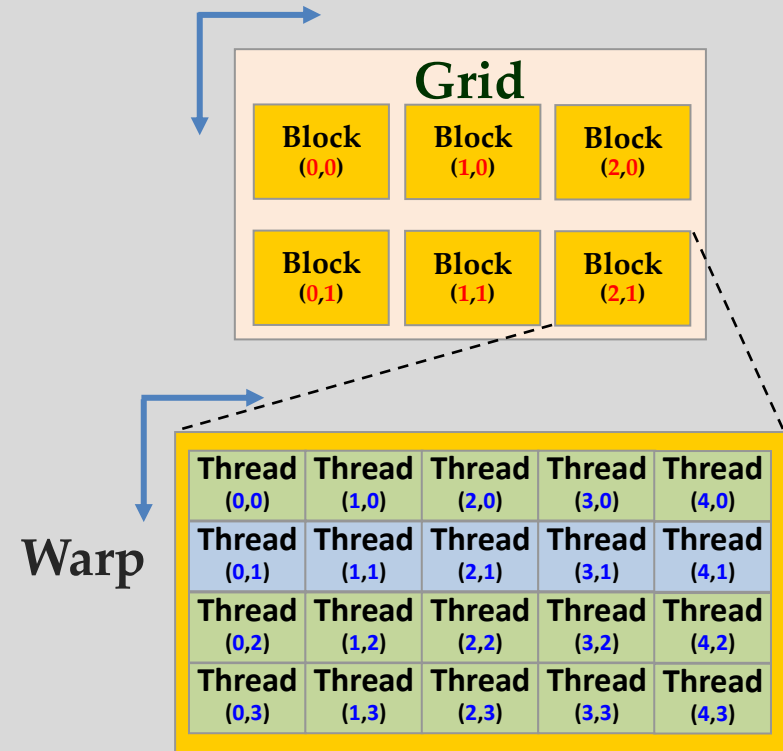


GPU Programming model

GPU Thread hierarchy

Block

- A group of threads that execute together.
- Threads within a block:
- Can cooperate via shared memory
- Can synchronize (e.g., in CUDA `syncthreads()`)
- Each block has an ID (`blockIdx`) and size (`blockDim`).

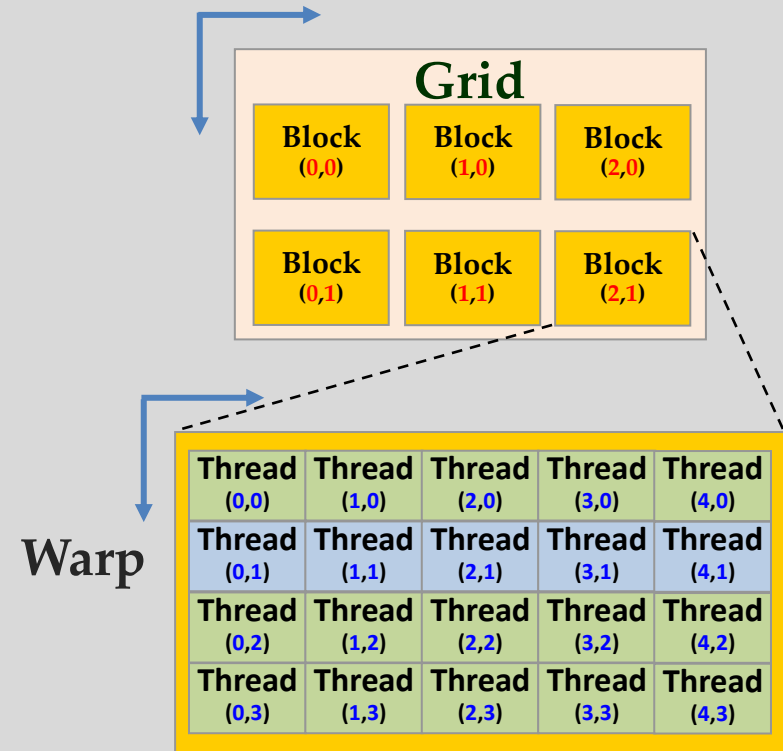


GPU Programming model

GPU Thread hierarchy

Grid

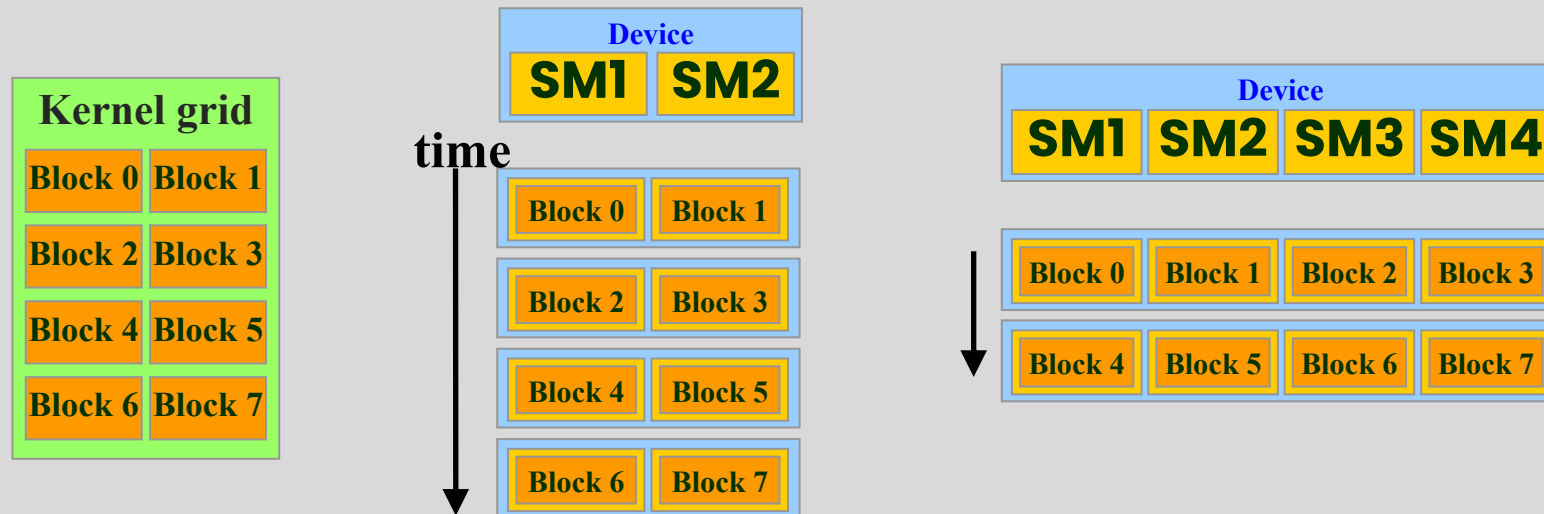
- A collection of blocks executing the same kernel.
- Defines the entire parallel computation.
- Blocks in different grids:
 - Cannot directly synchronize
 - Are scheduled independently.



GPU Programming model

GPU scalability

- the GPU system can execute thread blocks in any order relative to each other
- This flexibility enables to execute the same application code on hardware with different numbers of SMs



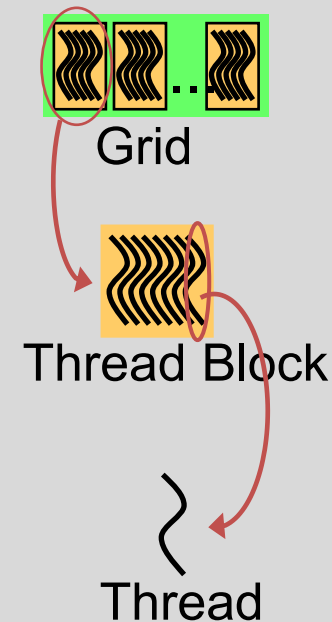
GPU Programming model

GPU execution model

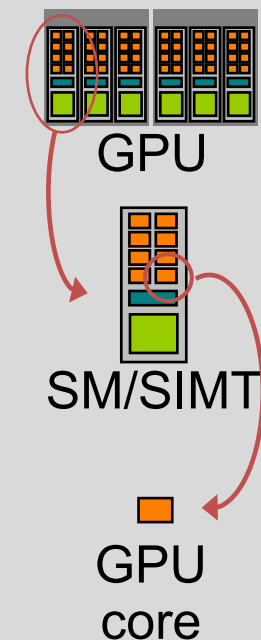
When a GPU kernel is invoked:

- Each thread block is assigned to an SM
- Each block execution is independent
- List of active blocks and assigns new blocks to SMs as they complete
- Constraints of thread block assignment
- once a block is assigned to an SM, it remains on that SM until the work for all threads in the block is completed

Software



Hardware



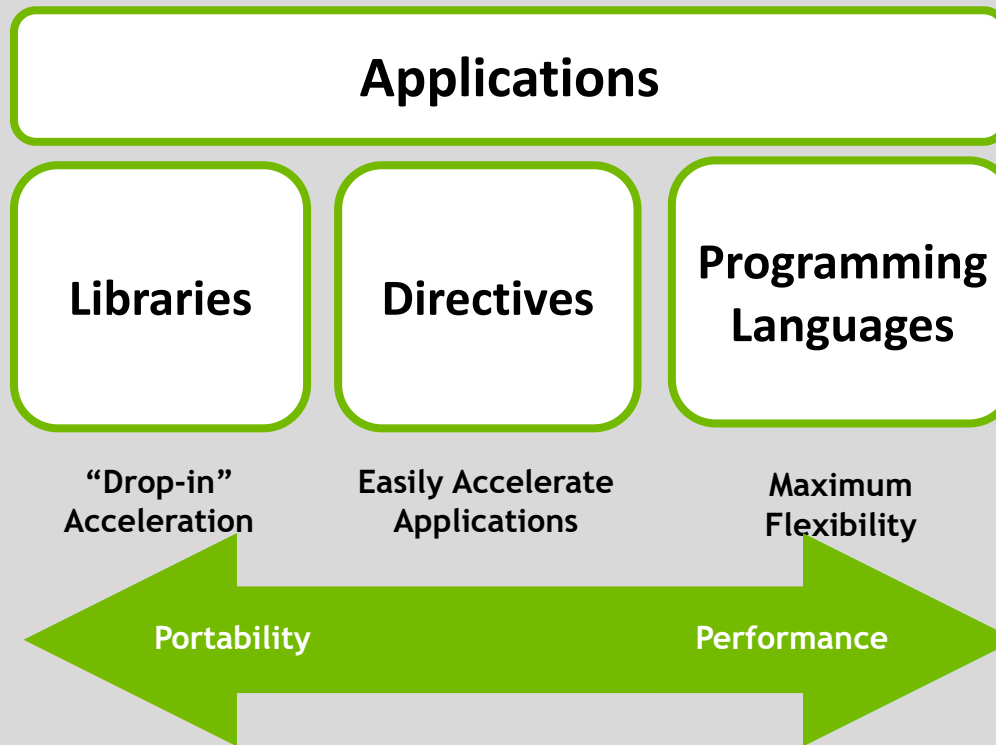
GPU Programming model

Programming paradigms

- **High level directives**
 - OpenACC
 - Thrust
- **High level programming languages**
 - C/ Cuda
 - OpenCL
 - GPU enabled libraries
- **Low level programming languages**
 - Assembly codes(*)

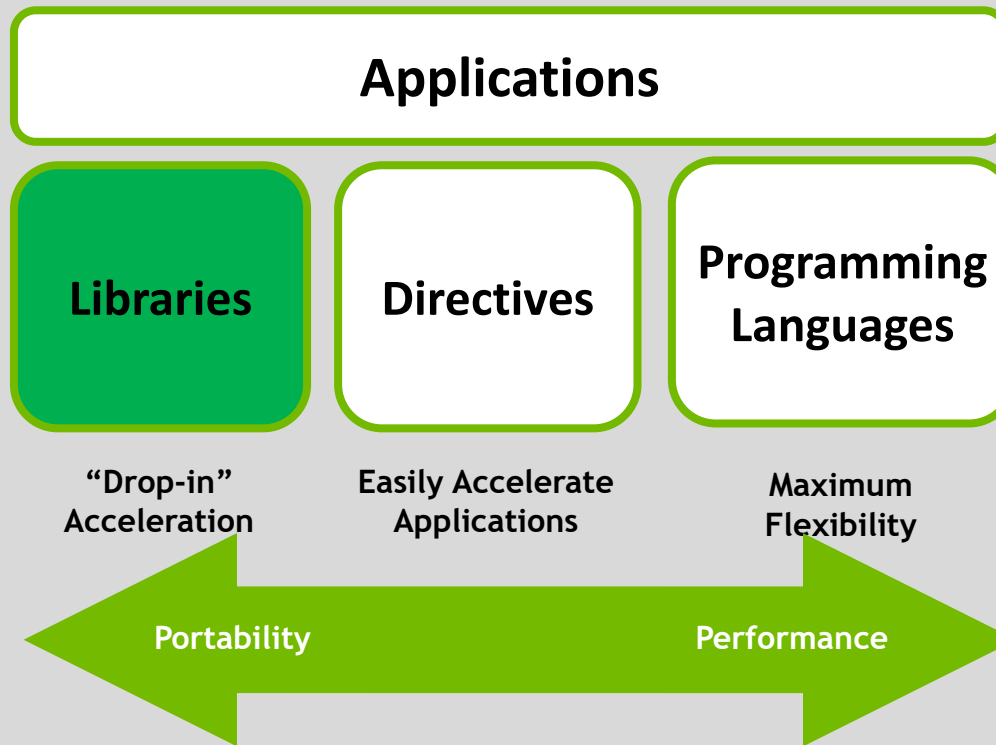
GPU Programming model

Programming paradigms



GPU Programming model

Programming paradigms



GPU Programming model

Programming paradigms

- Ease of use: Using libraries enables GPU acceleration without in-depth knowledge of GPU programming
- “Drop-in”: Many GPU-accelerated libraries follow standard APIs, thus enabling acceleration with minimal code changes
- Quality: Libraries offer high-quality implementations of functions encountered in a broad range of applications
- Performance: libraries are tuned by experts

GPU Programming model

Programming paradigms

Math Libraries

GPU-accelerated math libraries lay the foundation for compute-intensive applications in areas such as molecular dynamics, computational fluid dynamics, computational chemistry, medical imaging, and seismic exploration.



cuBLAS

GPU-accelerated basic linear algebra (BLAS) library.

[Learn More >](#)



cuFFT

GPU-accelerated library for Fast Fourier Transform implementations.

[Learn More >](#)



CUDA Math Library

GPU-accelerated standard mathematical function library.

[Learn More >](#)



cuRAND

GPU-accelerated random number generation (RNG).

[Learn More >](#)



cuSOLVER

GPU-accelerated dense and sparse direct solvers.

[Learn More >](#)



cuSPARSE

GPU-accelerated BLAS for sparse matrices.

[Learn More >](#)



cuTENSOR

GPU-accelerated tensor linear algebra library.

[Learn More >](#)



cuDSS

GPU-accelerated direct sparse solver library.

[Learn More >](#)



AmgX

GPU-accelerated linear solvers for simulations and implicit unstructured methods.

[Learn More >](#)

Partner Libraries

OpenCV

GPU-accelerated open-source library for computer vision, image processing, and machine learning, now supporting real-time operation.

[Learn More >](#)

FFmpeg

Open-source multimedia framework with a library of plug-ins for audio and video processing.

[Learn More >](#)

MAGMA

GPU-accelerated linear algebra routines for heterogeneous architectures, by Magma.

[Learn More >](#)

IMSL Fortran Numerical Library

GPU-accelerated open-source Fortran library with functions for math, signal and image processing, and statistics, by RogueWave.

[Learn More >](#)

CHOLMOD

GPU-accelerated functions for sparse direct solvers, included in the SuiteSparse linear algebra package, authored by Prof.

[Learn More >](#)

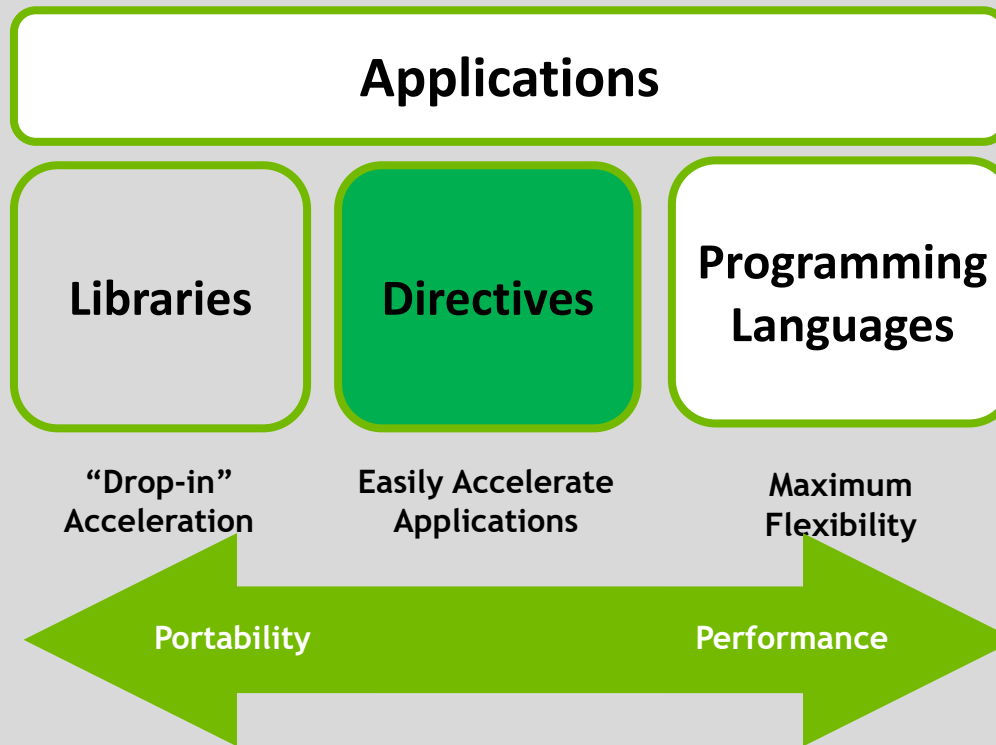
Triton Ocean SDK

Real-time visual simulation of oceans, water bodies in games, simulation, and training applications, by Triton.

[Learn More >](#)

GPU Programming model

Programming paradigms



GPU Programming model

Programming paradigms

- Added to serial source code (C/C++ or Fortran)
 - Manage loop parallelization
 - Manage data transfer between CPU and GPU memory
- Directives are formatted as comments
 - They don't interfere with serial execution
 - Maintains portability of original code
- Can be combined with explicit CUDA C/Fortran usage

GPU Programming model

Programming paradigms

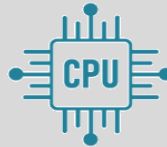
```
main() {  
    double pi = 0.0; long i;  
  
    for (i=0; i<N; i++)  
    {  
        double t = (double)((i+0.05)/N);  
        pi += 4.0/(1.0+t*t);  
    }  
  
    printf("pi = %f\n", pi/N);  
}
```

GPU Programming model

Programming paradigms

OpenMP

```
main() {  
    double pi = 0.0; long i;  
  
    #pragma omp parallel for reduction(+:pi)  
    for (i=0; i<N; i++)  
    {  
        double t = (double)((i+0.05)/N);  
        pi += 4.0/(1.0+t*t);  
    }  
  
    printf("pi = %f\n", pi/N);  
}
```



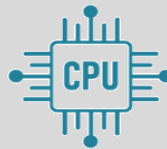
GPU Programming model

Programming paradigms

OpenACC
More Science, Less Programming

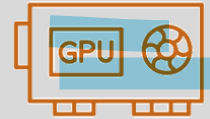
OpenMP

```
main() {  
    double pi = 0.0; long i;  
  
    #pragma omp parallel for reduction(+:pi)  
    for (i=0; i<N; i++)  
    {  
        double t = (double)((i+0.05)/N);  
        pi += 4.0/(1.0+t*t);  
    }  
  
    printf("pi = %f\n", pi/N);  
}
```



OpenACC

```
main() {  
    double pi = 0.0; long i;  
  
    #pragma acc parallel loop reduction(+:pi)  
    for (i=0; i<N; i++)  
    {  
        double t = (double)((i+0.05)/N);  
        pi += 4.0/(1.0+t*t);  
    }  
  
    printf("pi = %f\n", pi/N);  
}
```



GPU Programming model

Programming paradigms

Thrust



It looks like a library, with some primitives that help data transfer between cpu and gpu

```
#include <thrust/host_vector.h>
#include <thrust/device_vector.h>
#include <thrust/generate.h>
#include <thrust/sort.h>
#include <thrust/copy.h>
#include <cstdlib>

int main(void)
{
    // generate 32M random numbers on the host
    thrust::host_vector<int> h_vec(32 << 20);
    thrust::generate(h_vec.begin(), h_vec.end(), rand);

    // transfer data to the device
    thrust::device_vector<int> d_vec = h_vec;

    // sort data on the device (846M keys per second on GeForce GTX 480)
    thrust::sort(d_vec.begin(), d_vec.end());

    // transfer data back to host
    thrust::copy(d_vec.begin(), d_vec.end(), h_vec.begin());

    return 0;
}
```

Generate random numbers on the host and transfers them to the device where they are sorted.

GPU Programming model

Programming paradigms

Thrust

Generate random numbers on the host and transfers them to the device where they are sorted.



```
#include <thrust/host_vector.h>
#include <thrust/device_vector.h>
#include <thrust/generate.h>
#include <thrust/sort.h>
#include <thrust/copy.h>
#include <cstdlib>

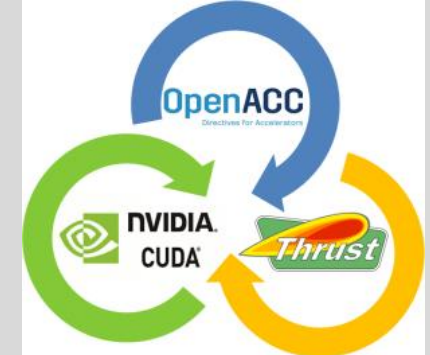
int main(void)
{
    // generate 32M random numbers on the host
    thrust::host_vector<int> h_vec(32 << 20);
    thrust::generate(h_vec.begin(), h_vec.end(), rand);

    // transfer data to the device
    thrust::device_vector<int> d_vec = h_vec;

    // sort data on the device (846M keys per second on GeForce GTX 480)
    thrust::sort(d_vec.begin(), d_vec.end());

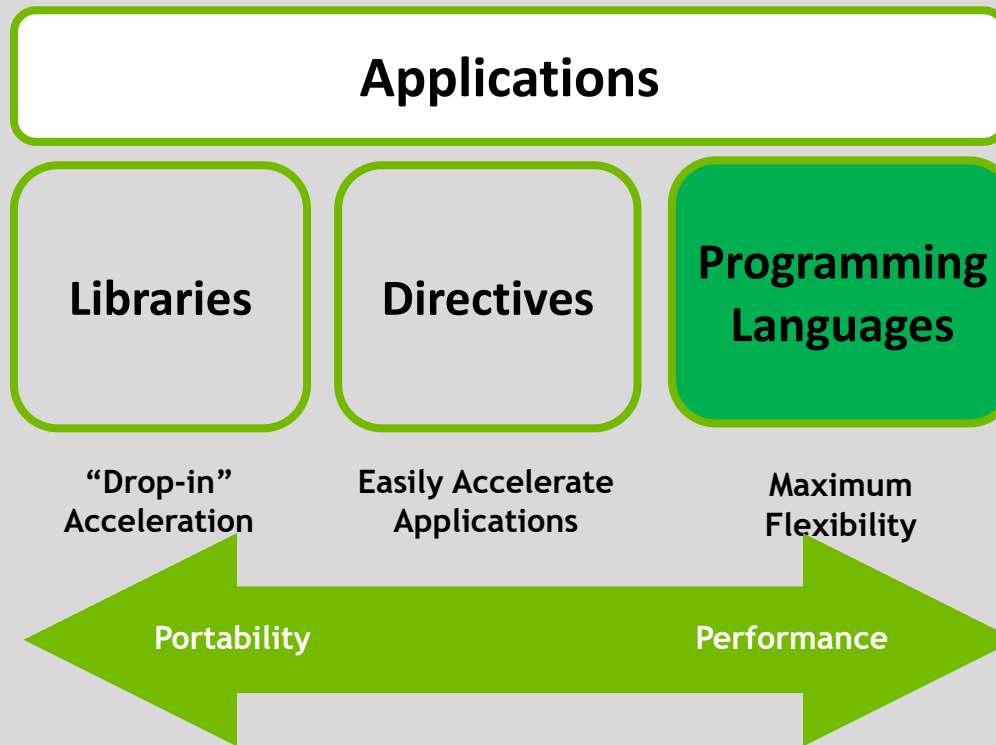
    // transfer data back to host
    thrust::copy(d_vec.begin(), d_vec.end(), h_vec.begin());

    return 0;
}
```



GPU Programming model

Programming paradigms



GPU Programming model

Programming paradigms

- **CUDA** (Compute Unified Device Architecture)



- a set of extensions to higher level programming language to use GPU as a coprocessor for heavy parallel task
- a developer toolkit to compile, debug, profile programs and run them easily in a heterogeneous systems



GPU Programming model

Programming paradigms

- **OpenCL** (Open Computing Language):
 - a standard **open-source** programming model developed by major brands of hardware manufacturers (Apple, Intel, AMD, nVIDIA, Imagination, ARM).
 - like CUDA, provides extensions to C/C++ and a developer toolkit
 - extensions for specific hardware (GPUs, FPGAs, MICs, etc)
 - it's very low level (verbose) programming



GPU Programming model

Programming paradigms

There are many other approaches and solutions, such as:

- SYCL (Khronos)
- HIP (AMD)
- OneAPI (Intel)
- DirectCompute (Microsoft)



... but current market is basically dominated by **CUDA** and **OpenCL**



Basic CUDA programming model

Basic CUDA programming model

How to use the GPU to speed up a program?

- Is there any level of parallelism?
- **Configure device**
- **Initialize the GPU's memory**
- **Define the total number of concurrent threads to be launch in the GPU**
- **Execute the program in the GPU**
- **Collect/retrieve the results to the Host**

Basic CUDA programming model

How to use the GPU to speed up a program?

- Is there any level of parallelism?
- **Configure device** (libraries, drivers, toolkit)
- **Initialize the GPUs memory** (application type, data)
- **Define the total number of concurrent threads to be launch in the GPU**
(Grid, Thread blocks, Warps, Threads)
- **Execute the program in the GPU** (one or more Kernels)
- **Collect/retrieve the results to the Host**

First example

Write a program for GPUs in CUDA to compute:

$$C[i] = A[i] + B[i]$$

With $i = 1,024$ elements.

1. Initialize data
2. Perform the Vector addition using CUDA
3. Write the resulting to the CPU

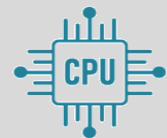
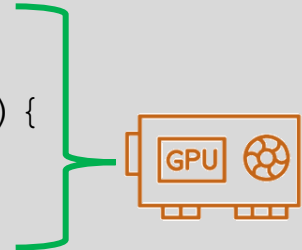
First example

Maybe darci un'occhiata

```
#include <cuda_runtime.h>
#include <iostream>

// CUDA KERNEL (GPU or device) -----
__global__ void vector_add(double *A, double *B, double *C, int N){
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        C[i] = A[i] + B[i]; }
}

// C code (CPU or host) -----
int main() {
    int N = 1 << 10;    // (2^10) = 1024 # of elements
    size_t size = N * sizeof(double);
    int threadsPerBlock = 256;
    int blocksPerGrid = 0;
    // Memory allocation in the host PC
    double *h_A = (double*)malloc(size);
    double *h_B = (double*)malloc(size);
    double *h_C = (double*)malloc(size);
```

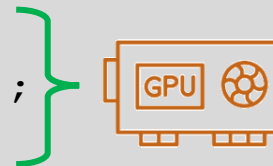
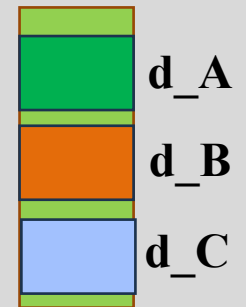
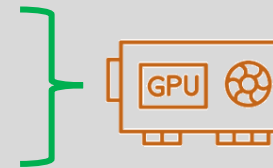
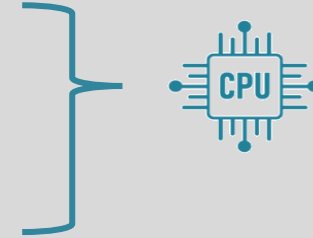


First example

```
// Vectors Init
for (int i = 0; i < N; i++) {
    h_A[i] = 1.0;
    h_B[i] = 2.0;
}

// allocate memory elements in the GPU or device
double *d_A, *d_B, *d_C;
cudaMalloc(&d_A, size);
cudaMalloc(&d_B, size);
cudaMalloc(&d_C, size);
// Copy from CPU MEM to GPU MEM
cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);
// Kernel configuration
blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;

// Kernel execution
vector_add<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);
```



First example

```
// Synchronize the threads execution
cudaDeviceSynchronize();

// Copy from GPU MEM to CPU MEM
cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

// Cleanup
cudaFree(d_A);
cudaFree(d_B);
cudaFree(d_C);
free(h_A);
free(h_B);
free(h_C);

return 0;
}
```

submit-cuda-sum.sh

```
#!/bin/bash
#SBATCH --job-name=cuda_sum          # job name
...
#SBATCH --partition=gpu_a40          # GPU Partition
#SBATCH --time=00:00:10              # Max EXE time (HH:MM:SS)
#SBATCH --output=output_%j.txt       # Output file (%j = job ID)
#SBATCH --mem-per-cpu=1G             # Max Memory
#SBATCH --gres=gpu:1                 # Uses 1 GPU
set -e #exit on error
# --- load modules ---
module purge
module load gcc
module load nvhpc-hpcx-cuda12/25.1
# --- print loaded modules ---
echo
module list
# --- program compile ---
nvcc cuda_sum.cu -o cuda_sum
#Program execution
./cuda_sum
```

output_12345.txt

Loading nvhpc-hpcx-cuda12/25.1

Loading requirement: hpcx

Currently Loaded Modulefiles:

1) gcc/12.4.0 2) hpcx <aL> 3) nvhpc-hpcx-cuda12/25.1

Key:

<module-tag> <aL>=auto-loaded

Numero di GPU CUDA rilevate: 1

----- RESULT -----

h_A[i] + h_B[i] = h_C[i] for i: 0 and 1023

h_A[0] = 1.000000

h_B[0] = 2.000000

h_C[0] = 3.000000

h_A[1023] = 1.000000

h_B[1023] = 2.000000

h_C[1023] = 3.000000